

The Transformer Architecture: A Self-Contained Explanation

Contents

1 Introduction and Motivation	1
2 Background: From Recurrence to Attention	2
3 The Transformer Architecture: High-Level Overview	2
4 Core Components in Detail	4
5 Why Self-Attention? Theoretical Analysis	7
6 Training Setup	9
7 Results	10
8 Simplified Architecture Diagram	12
9 Conclusion and Impact	13
References	14

1 Introduction and Motivation

1.1 The Problem: Sequence Transduction

Many important tasks in computing involve sequence transduction — converting one sequence of symbols into another. Machine translation is the canonical example: converting a sentence in English (the source sequence) into a sentence in French (the target sequence). Other examples include text summarization, speech recognition, and code generation.

Before 2017, the dominant approach to sequence transduction used recurrent neural networks (RNNs) — specifically variants called Long Short-Term Memory (LSTM) networks and Gated Recurrent Units (GRUs). These models process sequences one element at a time, maintaining a hidden state that summarizes everything seen so far.

1.2 The Limitation of Recurrence

The sequential nature of RNNs creates a fundamental bottleneck. To compute the representation at position t , the model must first compute the representation at position $t - 1$, which requires position $t - 2$, and so on. This inherently sequential computation prevents parallelization within a single training example. At longer sequence lengths, this becomes a severe practical limitation because memory constraints restrict how many examples can be processed simultaneously (batched).

1.3 The Innovation: Attention Without Recurrence

This paper introduces the Transformer, a model architecture that dispenses with recurrence and convolutions entirely, relying solely on attention mechanisms to draw global dependencies between input and output positions. The key advantages are:

- Parallelization: All positions can be processed simultaneously
- Training speed: Significantly less time to train
- Quality: State-of-the-art results on translation benchmarks

2 Background: From Recurrence to Attention

2.1 Recurrent Models

In a recurrent model, the hidden state at time t is computed as:

$$h_t = f(h_{t-1}, x_t)$$

where x_t is the input at position t and f is a learned function (typically an LSTM or GRU cell). This creates a chain of dependencies of length n for a sequence of length n .

2.2 Convolutional Models

Models like ByteNet and ConvS2S use convolutional neural networks (CNNs) instead of recurrence. CNNs can process all positions in parallel, but relating two distant positions requires stacking many layers. With kernel width k , the number of layers needed grows as $O(n/k)$ for standard convolutions or $O(\log_k n)$ for dilated convolutions. This increases the path length — the number of steps a signal must travel through the network — making long-range dependencies harder to learn.

2.3 Attention Mechanisms

Attention allows a model to directly access any position in a sequence when computing a representation for another position, regardless of distance. Before the Transformer, attention was typically used in conjunction with recurrent or convolutional networks (e.g., an RNN encoder with attention over its hidden states). The Transformer’s insight: attention alone is sufficient — no recurrence or convolution needed.

3 The Transformer Architecture: High-Level Overview

The Transformer follows the standard encoder-decoder structure:

1. Encoder: Maps an input sequence $(x_1, \dots, x_n)$ to a sequence of continuous representations $z = (z_1, \dots, z_n)$
2. Decoder: Generates an output sequence $(y_1, \dots, y_m)$ one element at a time, using the encoder’s output z and previously generated symbols

Both encoder and decoder are composed of stacks of identical layers. Each layer contains two core components: - Multi-head self-attention (the primary mixing mechanism) - Position-wise feed-forward networks (pointwise processing)

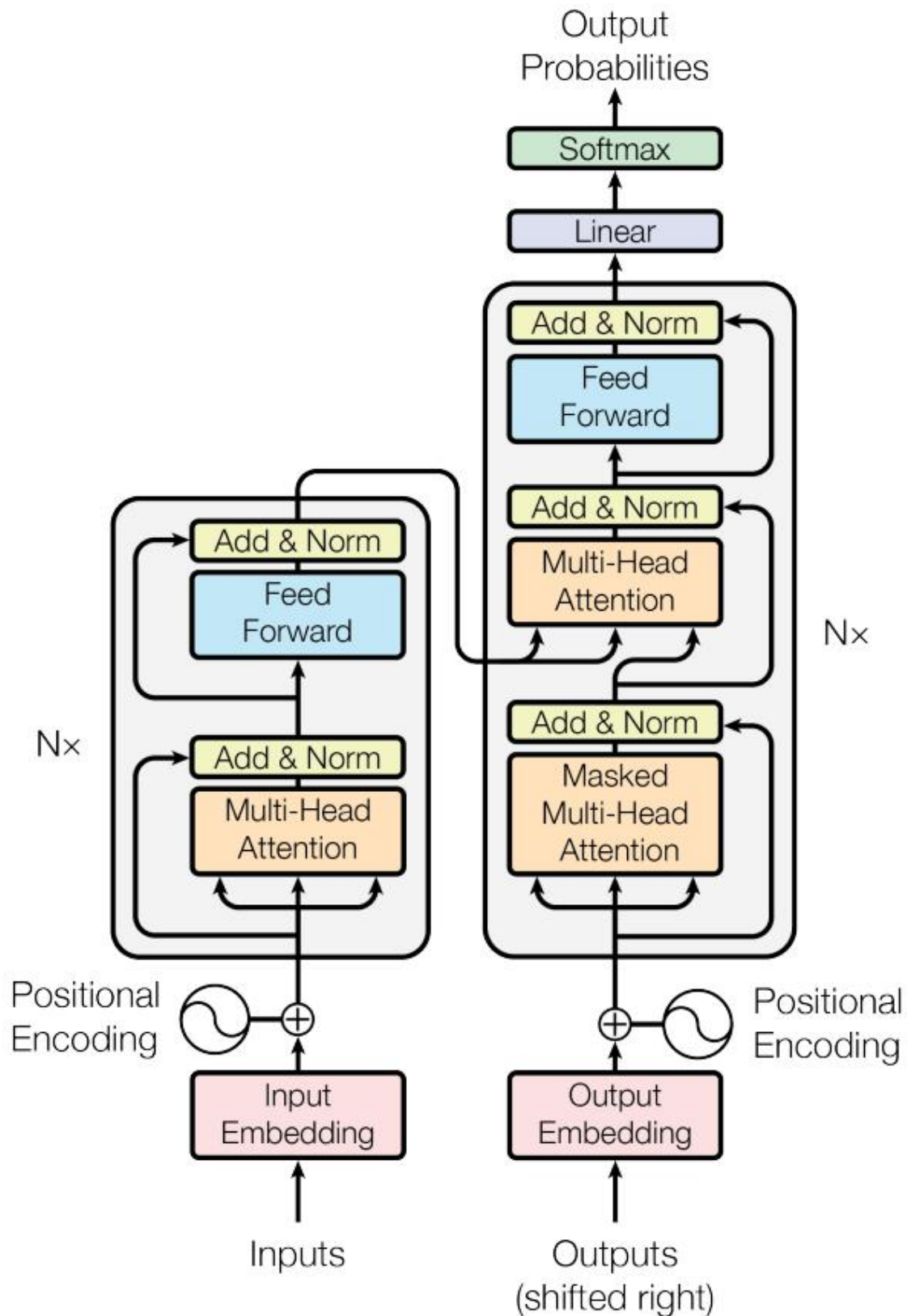


Figure 1: The Transformer model architecture. The left half shows the encoder stack (6 layers), the right half shows the decoder stack (6 layers). Each layer contains multi-head attention and feed-forward sub-layers with residual connections and layer normalization.

The encoder processes the full input sequence at once. The decoder generates tokens autoregressively — at step t , it can only attend to positions $< t$ in the target sequence (enforced by masking).

4 Core Components in Detail

4.1 Scaled Dot-Product Attention

4.1.1 The Attention Operation An attention function maps a query vector q and a set of key-value pairs $\{(k_i, v_i)\}$ to an output vector. The output is a weighted sum of the values, where the weight for each value is determined by a compatibility function between the query and the corresponding key.

In the Transformer, queries, keys, and values are packed into matrices Q, K, V (each row is one position’s vector). The attention computation is:

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V \quad (1)$$

What this computes, step by step:

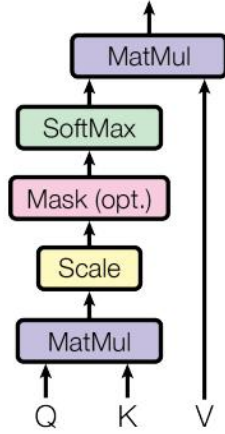
1. QK^T : Matrix multiplication computes dot products between every query and every key. The (i, j) entry is $q_i \cdot k_j$ — a measure of how relevant position j is to position i .
2. Division by $\sqrt{d_k}$: Scales the dot products (explained below).
3. softmax: Converts scores to probabilities summing to 1 across keys for each query. This gives the attention weights.
4. Multiplication by V : Weighted sum of value vectors using the attention weights.

The result is a matrix where row i is the output for query position i — a context-aware representation that aggregates information from all positions, weighted by relevance.

4.1.2 Why Scale by $\sqrt{d_k}$? For large key dimension d_k , dot products grow large in magnitude (variance scales with d_k). Large values push the softmax function into regions with extremely small gradients, making training difficult. Scaling by $\sqrt{d_k}$ counteracts this effect.

Illustration: If q and k have independent components with mean 0 and variance 1, their dot product has variance d_k . Without scaling, the softmax inputs would have standard deviation $\sqrt{d_k}$.

Scaled Dot-Product Attention



Multi-Head Attention

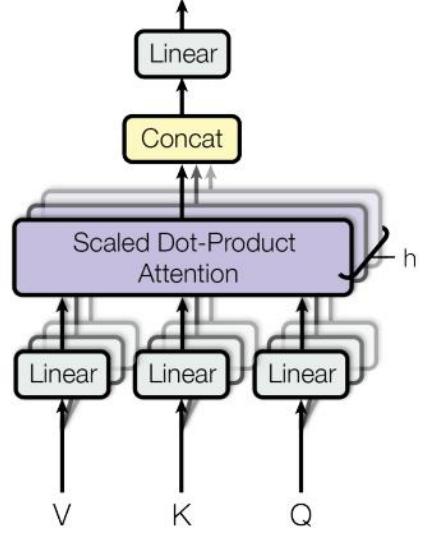


Figure 2: Left: Scaled Dot-Product Attention computation flow. Right: Multi-Head Attention runs several attention layers in parallel, then concatenates and projects their outputs.

4.2 Multi-Head Attention

4.2.1 Motivation A single attention head computes one weighted average over positions. But different aspects of a sequence may require different attention patterns — e.g., one head might attend to syntactic dependencies, another to semantic relations. Averaging everything in one head can blur these distinct patterns.

4.2.2 Mechanism Instead of one attention function with dimension d_{model} , we project queries, keys, and values h times with different learned linear projections to dimensions d_k, d_k, d_v :

$$\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$$

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W^O \quad (2)$$

Where: - $W_i^Q \in \mathbb{R}^{d_{\text{model}} \times d_k}$, $W_i^K \in \mathbb{R}^{d_{\text{model}} \times d_k}$, $W_i^V \in \mathbb{R}^{d_{\text{model}} \times d_v}$ are learned projection matrices for head i - $W^O \in \mathbb{R}^{hd_v \times d_{\text{model}}}$ is the output projection - In the paper: $h = 8$, $d_k = d_v = d_{\text{model}}/h = 64$

The total computational cost is similar to single-head attention with full dimensionality, but the model can now jointly attend to information from different representation subspaces at different positions.

4.3 Three Uses of Attention in the Transformer

Use Case	Queries From	Keys/Values From	Purpose
Encoder self-attention	Previous encoder layer	Previous encoder layer	Each encoder position attends to all encoder positions
Decoder self-attention	Previous decoder layer	Previous decoder layer	Each decoder position attends to earlier decoder positions (masked)
Encoder-decoder attention	Previous decoder layer	Encoder output	Each decoder position attends to all encoder positions

Decoder masking: To preserve autoregressive generation (position t predicts token $t + 1$ using only tokens $\leq t$), the decoder self-attention masks out (sets to $-\infty$) attention weights for future positions before the softmax.

4.4 Position-Wise Feed-Forward Networks (FFN)

Each layer (in both encoder and decoder) contains a fully connected feed-forward network applied independently to each position:

$$\text{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2 \quad (3)$$

- Two linear transformations with a ReLU activation ($\max(0, \cdot)$) in between
- Input/output dimension: $d_{\text{model}} = 512$
- Inner layer dimension: $d_{\text{ff}} = 2048$
- Equivalent to two 1D convolutions with kernel size 1
- Parameters differ across layers but are shared across positions within a layer

This component processes the information gathered by attention at each position independently.

4.5 Residual Connections and Layer Normalization

Each sub-layer (attention or FFN) is wrapped in a residual connection followed by layer normalization:

$$\text{LayerNorm}(x + \text{Sublayer}(x))$$

- Residual connection: Adds the sub-layer input to its output, enabling gradient flow in deep networks

- Layer normalization: Normalizes activations across the feature dimension for each position independently (unlike batch normalization which normalizes across the batch)
- All sub-layers and embedding layers output dimension $d_{\text{model}} = 512$ to facilitate residuals

4.6 Embeddings and Softmax

- Token embeddings: Learned mappings from vocabulary indices to d_{model} -dimensional vectors
- Weight tying: The same weight matrix is used for:
 1. Input token embeddings (encoder)
 2. Output token embeddings (decoder)
 3. Pre-softmax linear transformation (projecting decoder output to vocabulary logits)
- Embedding weights are multiplied by $\sqrt{d_{\text{model}}}$ to scale them appropriately

4.7 Positional Encoding

Since the Transformer has no recurrence or convolution, it has no inherent notion of sequence order. Positional encodings inject position information by adding fixed vectors to token embeddings.

The paper uses sinusoidal positional encodings:

$$PE_{(pos, 2i)} = \sin\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right)$$

$$PE_{(pos, 2i+1)} = \cos\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right)$$

Where: - pos = position index (0, 1, 2, ...) - i = dimension index (0 to $d_{\text{model}}/2 - 1$) - Each dimension is a sinusoid with wavelength forming a geometric progression from 2π to $10000 \cdot 2\pi$

Key property: For any fixed offset k , PE_{pos+k} can be represented as a linear function of PE_{pos} . This allows the model to easily learn relative position relationships.

Learned positional embeddings were also tested and performed nearly identically, but sinusoidal encodings may extrapolate better to longer sequences than seen during training.

5 Why Self-Attention? Theoretical Analysis

The paper compares self-attention to recurrent and convolutional layers on three desiderata for mapping a sequence $(x_1, \dots, x_n)$ to $(z_1, \dots, z_n)$ with $x_i, z_i \in \mathbb{R}^d$:

5.1 Computational Complexity Per Layer

Layer Type	Complexity
Self-Attention	$O(n^2 \cdot d)$
Recurrent	$O(n \cdot d^2)$
Convolutional	$O(k \cdot n \cdot d^2)$

Self-attention is faster than recurrent layers when $n < d$ (true for typical NLP sequences with word-piece/byte-pair representations).

5.2 Sequential Operations (Parallelizability)

Layer Type	Sequential Operations
Self-Attention	$O(1)$
Recurrent	$O(n)$
Convolutional	$O(1)$

Self-attention and convolutions allow full parallelization across positions; recurrence does not.

5.3 Maximum Path Length (Long-Range Dependencies)

Layer Type	Maximum Path Length
Self-Attention	$O(1)$
Recurrent	$O(n)$
Convolutional	$O(\log_k n)$

Shorter paths mean gradients flow more easily between distant positions, making long-range dependencies easier to learn.

5.4 Summary Table (Reproduced from Paper)

Layer Type	Complexity per Layer	Sequential Operations	Maximum Path Length
Self-Attention	$O(n^2 \cdot d)$	$O(1)$	$O(1)$
Recurrent	$O(n \cdot d^2)$	$O(n)$	$O(n)$
Convolutional	$O(k \cdot n \cdot d^2)$	$O(1)$	$O(\log_k n)$
Self-Attention (restricted)	$O(r \cdot n \cdot d)$	$O(1)$	$O(n/r)$

Table 1: Comparison of layer types. n = sequence length, d = representation dimension, k = kernel size, r = neighborhood size for restricted attention.

Interpretability bonus: Attention weights can be inspected. Individual heads learn distinct patterns, many corresponding to syntactic or semantic structure (e.g., subject-verb agreement, coreference).

6 Training Setup

6.1 Data and Tokenization

Task	Dataset	Size	Tokenization	Vocabulary
EN-DE	WMT 2014	4.5M sentence pairs	Byte-Pair Encoding (BPE)	~37,000 shared tokens
EN-FR	WMT 2014	36M sentence pairs	WordPiece	32,000 tokens

Batches grouped by approximate sequence length; each batch contains ~25,000 source + 25,000 target tokens.

6.2 Hardware and Schedule

- Hardware: 8 NVIDIA P100 GPUs
- Base model: 100,000 steps (~12 hours, 0.4 sec/step)
- Big model: 300,000 steps (~3.5 days, 1.0 sec/step)

6.3 Optimizer and Learning Rate Schedule

Adam optimizer with $\beta_1 = 0.9$, $\beta_2 = 0.98$, $\epsilon = 10^{-9}$.

Learning rate schedule:

$$\text{lr} = d_{\text{model}}^{-0.5} \cdot \min(\text{step_num}^{-0.5}, \text{step_num} \cdot \text{warmup_steps}^{-1.5}) \quad (4)$$

- Linear warmup for first 4000 steps
- Then decay proportional to inverse square root of step number
- $d_{\text{model}}^{-0.5}$ scaling accounts for embedding magnitude

6.4 Regularization

1. Residual Dropout ($P_{\text{drop}} = 0.1$ for base, 0.3 for big): Applied to sub-layer outputs before residual addition, and to embedding + positional encoding sums
2. Label Smoothing ($\epsilon_{ls} = 0.1$): During training, target distribution is smoothed: $(1 - \epsilon)$ on true token, $\epsilon/(V - 1)$ on others. Improves BLEU despite hurting perplexity.

7 Results

7.1 Machine Translation Benchmarks

Model	EN-DE BLEU	EN-FR BLEU	Training Cost (FLOPs) EN-DE	Training Cost (FLOPs) EN-FR
ByteNet [15]	23.75	—	—	—
Deep-Att + Po-sUnk [32]	—	39.2	—	1.0×10^{20}
GNMT + RL [31]	24.6	39.92	2.3×10^{19}	1.4×10^{20}
ConvS2S [8]	25.16	40.46	9.6×10^{18}	1.5×10^{20}
MoE [26]	26.03	40.56	2.0×10^{19}	1.2×10^{20}
GNMT + RL Ensemble [31]	26.30	41.16	1.8×10^{20}	1.1×10^{21}
ConvS2S Ensemble [8]	26.36	41.29	7.7×10^{19}	1.2×10^{21}
Transformer (base)	27.3	38.1	3.3×10^{18}	—
Transformer (big)	28.4	41.0	2.3×10^{19}	—

Table 2: Translation quality (BLEU) and training cost comparison. Higher BLEU is better. Transformer (big) outperforms all previous single models and even most ensembles at a fraction of the training cost.

Key results: - EN-DE: Transformer (big) achieves 28.4 BLEU, >2 BLEU above previous best (including ensembles) - EN-FR: Transformer (big) achieves 41.0 BLEU, best single-model result, <1/4 training cost of previous SOTA - Base model already surpasses all previously published models/ensembles

Inference details: Beam search (beam size 4), length penalty $\alpha = 0.6$, max output length = input length + 50. Base model: average last 5 checkpoints. Big model: average last 20 checkpoints.

7.2 Model Variations (Ablation Study)

All experiments on EN-DE development set (newstest2013), single model, no checkpoint averaging.

Variation	N	d_{model}	d_{ff}	h	d_k	d_v	P_{drop}	ϵ_{ls}	PPL (dev)	BLEU (dev)	Params (M)
Base	6	512	2048	8	64	64	0.1	0.1	4.92	25.8	65
(A)				1	512	512			5.29	24.9	
Heads:											
1											
(A)				4	128	128			5.00	25.5	
Heads:											
4											
(A)				16	32	32			4.91	25.8	
Heads:											
16											
(A)				32	16	16			5.01	25.4	
Heads:											
32											
(B)					16				5.16	25.1	58
$d_k =$											
16											
(B)					32				5.01	25.4	60
$d_k =$											
32											
(C)	2								6.11	23.7	36
Lay-											
ers:											
2											
(C)	4								5.19	25.3	50
Lay-											
ers:											
4											
(C)	8								4.88	25.5	80
Lay-											
ers:											
8											
(C)		256			32	32			5.75	24.5	28
$d_{\text{model}} =$											
256											
(C)		1024			128	128			4.66	26.0	168
$d_{\text{model}} =$											
1024											
(C)			1024						5.12	25.4	53
$d_{\text{ff}} =$											
1024											
(C)			4096						4.75	26.2	90
$d_{\text{ff}} =$											
4096											
(D)							0.0		5.77	24.6	
No											
dropout											
(D)							0.2		4.95	25.5	
High											
dropout											

Variation	N	d_{model}	d_{ff}	h	d_k	d_v	P_{drop}	ϵ_{ls}	PPL (dev)	BLEU (dev)	Params (M)
(D) No label smooth- ing								0.0	4.67	25.3	
(D) High label smooth- ing								0.2	5.47	25.7	
(E) Learned pos. em- bed- dings									4.92	25.7	
Big	6	1024	4096	16			0.3		4.33	26.4	213

Table 3: Ablation study results. Unlisted values match base model. PPL = perplexity (lower is better). BLEU = development set score (higher is better).

Key findings: - (A) Number of heads: Single head is 0.9 BLEU worse; too many heads also hurts. 8 heads is optimal. - (B) Key dimension d_k : Reducing d_k hurts quality, suggesting dot-product compatibility benefits from higher dimensionality. - (C) Model size: Deeper, wider models improve quality (diminishing returns). $d_{\text{ff}} = 4096$ helps significantly. - (D) Regularization: Dropout is crucial (1.2 BLEU drop without). Label smoothing helps BLEU despite hurting perplexity. - (E) Positional encoding: Learned embeddings perform nearly identically to sinusoidal.

8 Simplified Architecture Diagram

ENCODER (×6 layers)

Input Embeddings
+ Positional Encoding

Multi-Head Self-Attention
(all positions attend)

Add & Norm

DECODER (×6 layers)

Output Embeddings
+ Positional Encoding

Masked Multi-Head Self-Attn
(causal: only past positions)

Add & Norm

Feed-Forward Network
(position-wise)

Encoder-Decoder Attention
(queries=dec, keys/vals=enc)

Add & Norm

Add & Norm

(to next layer)

(to next layer)

Feed-Forward Network

Add & Norm

Linear + Softmax
(vocabulary projection)

Each “Add & Norm” = residual connection + layer normalization. Encoder output feeds into every decoder layer’s encoder-decoder attention.

9 Conclusion and Impact

The Transformer established that attention alone — without recurrence or convolution — is sufficient for state-of-the-art sequence transduction. Its advantages:

1. Parallelization: $O(1)$ sequential operations vs $O(n)$ for RNNs
2. Long-range dependencies: Constant path length between any two positions
3. Training efficiency: 3.5 days on 8 GPUs vs weeks for prior models
4. Quality: New SOTA on WMT 2014 EN-DE (28.4 BLEU) and EN-FR (41.0 BLEU)

The architecture has since become the foundation for nearly all large language models (BERT, GPT, T5, etc.) and has been extended to vision, audio, and multimodal tasks. The core insight — that global pairwise interaction via attention can replace sequential recurrence — reshaped the field of deep learning for sequences.

References

- [1] Ba, Kiros, Hinton. Layer Normalization. 2016.
- [2] Bahdanau, Cho, Bengio. Neural Machine Translation by Jointly Learning to Align and Translate. 2014.
- [3] Britz et al. Massive Exploration of Neural Machine Translation Architectures. 2017.
- [4] Cheng, Dong, Lapata. LSTM for Machine Reading. 2016.
- [5] Cho et al. Learning Phrase Representations using RNN Encoder-Decoder. 2014.
- [6] Chollet. Xception: Deep Learning with Depthwise Separable Convolutions. 2016.
- [7] Chung et al. Empirical Evaluation of Gated Recurrent Neural Networks. 2014.
- [8] Gehring et al. Convolutional Sequence to Sequence Learning. 2017.
- [9] Graves. Generating Sequences with RNNs. 2013.
- [10] He et al. Deep Residual Learning. 2016.
- [11] Hochreiter et al. Gradient Flow in Recurrent Nets. 2001.
- [12] Hochreiter, Schmidhuber. Long Short-Term Memory. 1997.
- [13] Jozefowicz et al. Exploring the Limits of Language Modeling. 2016.
- [14] Kaiser, Sutskever. Neural GPUs Learn Algorithms. 2016.
- [15] Kalchbrenner et al. Neural Machine Translation in Linear Time. 2017.
- [16] Kim et al. Structured Attention Networks. 2017.
- [17] Kingma, Ba. Adam: A Method for Stochastic Optimization. 2015.
- [18] Kuchaiev, Ginsburg. Factorization Tricks for LSTM Networks. 2017.
- [19] Lin et al. Structured Self-Attentive Sentence Embedding. 2017.
- [20] Kaiser, Bengio. Can Active Memory Replace Attention? 2016.
- [21] Luong, Pham, Manning. Effective Approaches to Attention-Based NMT. 2015.
- [22] Parikh et al. A Decomposable Attention Model. 2016.
- [23] Paulus, Xiong, Socher. Deep Reinforced Model for Abstractive Summarization. 2017.
- [24] Press, Wolf. Using the Output Embedding to Improve Language Models. 2016.
- [25] Sennrich, Haddow, Birch. Neural Machine Translation of Rare Words with Subword Units. 2015.
- [26] Shazeer et al. Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer. 2017.
- [27] Srivastava et al. Dropout: A Simple Way to Prevent Neural Networks from Overfitting. 2014.
- [28] Sukhbaatar et al. End-to-End Memory Networks. 2015.
- [29] Sutskever, Vinyals, Le. Sequence to Sequence Learning with Neural Networks. 2014.
- [30] Szegedy et al. Rethinking the Inception Architecture. 2015.
- [31] Wu et al. Google’s Neural Machine Translation System. 2016.
- [32] Zhou et al. Deep Recurrent Models with Fast-Forward Connections for NMT. 2016.